

# GPU Programming Models

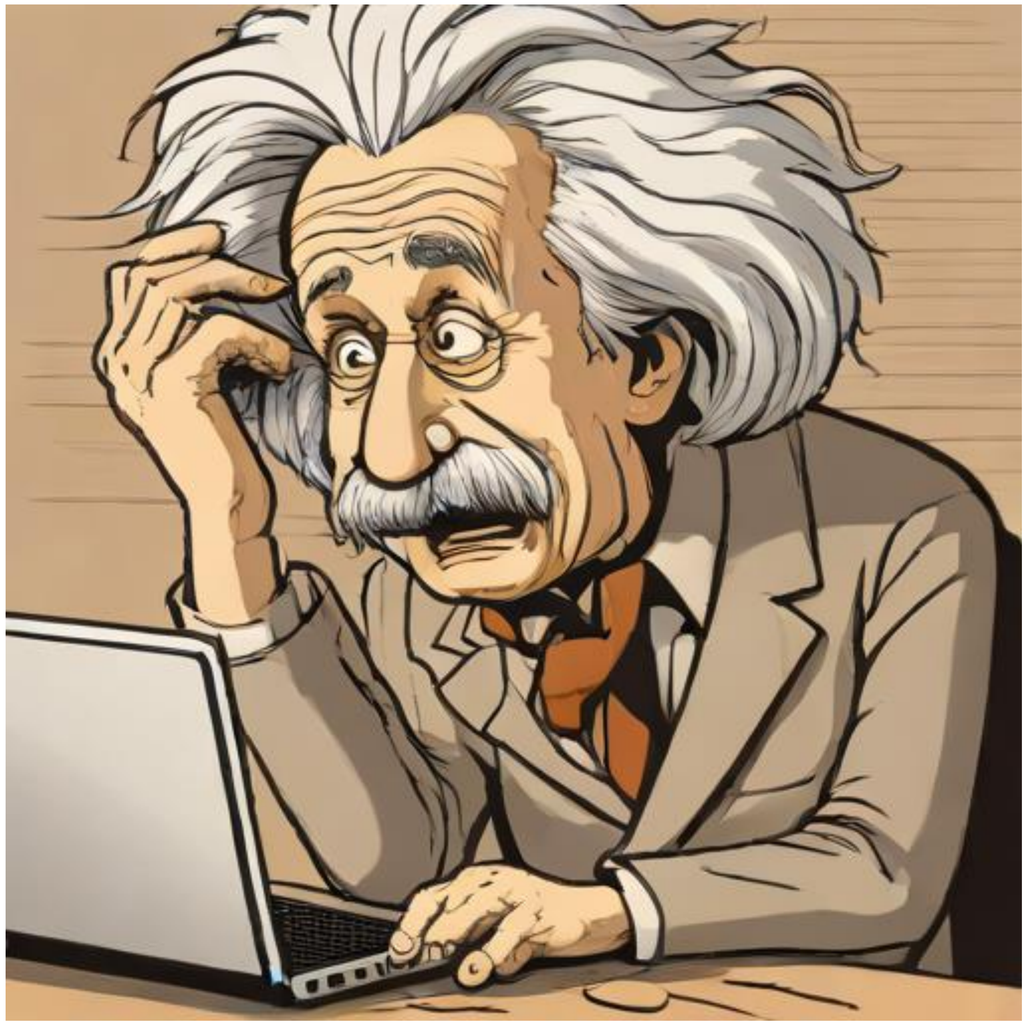
IEEE Cluster 2025

# Future-proof GPU programming for HPC and AI

# Dispelling Some Myths

- GPU Programming...
  - …is hard
  - …is time-consuming
  - …isn't portable
  - …requires a special language or extensions
  - …can only be done in C++
  - …is only for ninja programmers

FALSE



# Programming the NVIDIA platform

CPU, GPU, and Network



### ACCELERATED STANDARD LANGUAGES

ISO C++, ISO Fortran

```
std::transform(par, x, x+n, y, y,
    [=](float x, float y){ return y +
a*x; }
);
```

```
do concurrent (i = 1:n)
   y(i) = y(i) + a*x(i)
enddo
```

```
import cunumeric as np
…
def saxpy(a, x, y):
    y[:] += a*x
```

### INCREMENTAL PORTABLE OPTIMIZATION

OpenACC, OpenMP

```
#pragma acc data copy(x,y) {
...
std::transform(par, x, x+n, y, y,
    [=](float x, float y){
        return y + a*x;
});
...
}
```

```
#pragma omp target data map(x,y) {
...
std::transform(par, x, x+n, y, y,
    [=](float x, float y){
        return y + a*x;
});
...
}
```

### PLATFORM SPECIALIZATION

CUDA

```
__global__
void saxpy(int n, float a,
           float *x, float *y) {
  int i = blockIdx.x*blockDim.x +
          threadIdx.x;
  if (i < n) y[i] += a*x[i];
}

int main(void) {
  ...
  cudaMemcpy(d_x, x, ...);
  cudaMemcpy(d_y, y, ...);

  saxpy<<<(N+255)/256,256>>>(...);

  cudaMemcpy(y, d_y, ...);
```

### ACCELERATION LIBRARIES

| Core | Math | Communication | Data Analytics | AI | Quantum |
|---|---|---|---|---|---|



# Choosing A Programming Model

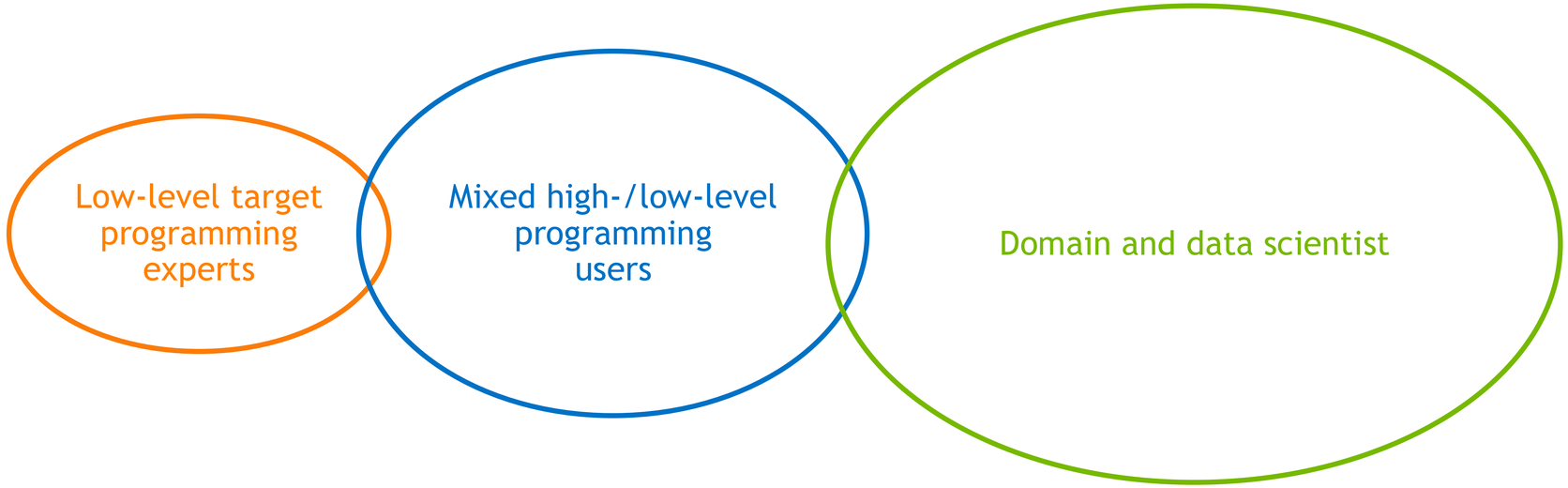
There can be ~~only~~ ***more than*** one.

| Libraries | Standard Languages | Compiler Directives | CUDA Languages |
|---|---|---|---|
| • Accelerate common operations with little/no code changes.<br>• Expert-tuned performance.<br>• Forward support guarantees. | • Strong cross-platform support.<br>• Single source code for multiple platforms.<br>• Reduced learning curve. | • High cross-platform support.<br>• Single source code for multiple platforms.<br>• Reduced learning curve.<br>• Additional programmer control. | • Exposes full GPU capabilities.<br>• Trades portability for performance.<br>• Distinct GPU/CPU code paths.<br>• Full programmer control. |
| Programmer Productivity | | | Programmer Control |

By design these approaches are interoperable so developers can choose the right balance for their needs.

# Programming Models Target Audience

Three categories of users




Parallelism, data management, and synchronization controlled by user

Parallelism, data management, and synchronization controlled by system

# Accelerated Standard Languages

Parallel performance for wherever your code runs

**ISO C++**

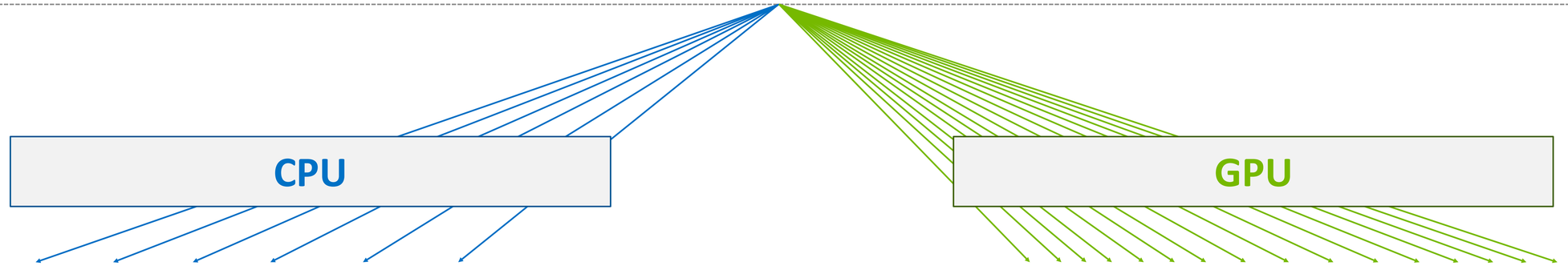
```
std::transform(par, x, x+n, y,
   y,[=](float x, float y){
      return y + a*x;
    }
);
```

**ISO Fortran**

```
do concurrent (i = 1:n)
   y(i) = y(i) + a*x(i)
enddo
```

**Python**

```
import cunumeric as np
…
def saxpy(a, x, y):
    y[:] += a*x
```




```
nvc++ -stdpar=multicore
nvfortran –stdpar=multicore
legate –cpus 16 saxpy.py
```

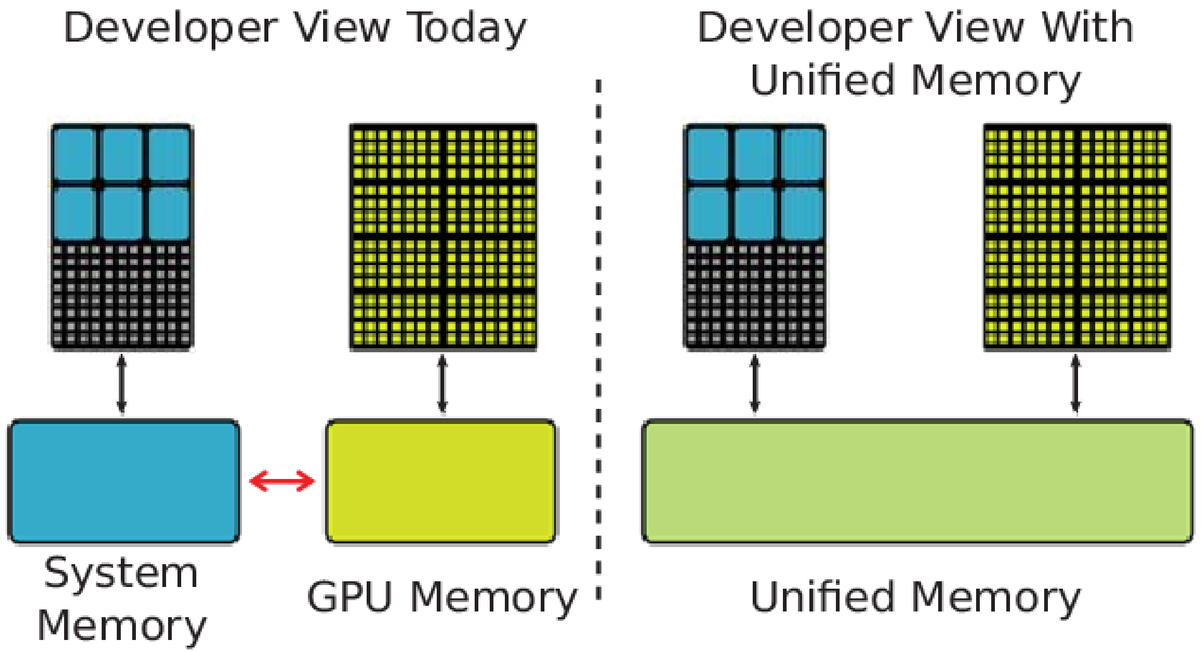
```
nvc++ -stdpar=gpu
nvfortran –stdpar=gpu
legate –gpus 1 saxpy.py
```

# Unified Memory

## Dramatically Lowering Developer Effort

CUDA **Unified Memory** allows a program to dynamically allocate data that can be accessed from the CPU or from the GPU, with the same pointer and address

- Introduced with CUDA 6 / Kepler GK10x
- Leverages UVA (Unified Virtual Addressing) beyond GPU memory pools
- CUDA driver managed page migration
  CPU <--> GPU on-demand when data is requested (*first touch*)
- Can be controlled / tweaked with prefetching APIs and by define User Policy
  - `ReadMostly`
  - `PreferredLocation`
  - `AccessedBy`



# CUDA Unified Memory Timeline

## The History of Unified Memory in CUDA

### 2011

**Unified Virtual Addresses**

CUDA 4: Unified addresses between CPU and GPU memories.

### 2014

**Unified Memory**

CUDA 6: Unified (Virtual) Memory (Managed Memory), one pointer for data, implicit migration

### 2017

**Address Translation Service**

CUDA 9: Unified page table between Power 9 CPU and V100 GPU and NVLINK2 interconnect.

### 2017

**HMM**

Heterogeneous Memory Management merged into Linux kernel.

### 2023

**Grace Hopper**

Grace Hopper supports true unified memory (ATS) with NVLINK4. Extended GPU Memory (EGM). Intelligent pre-fetching.

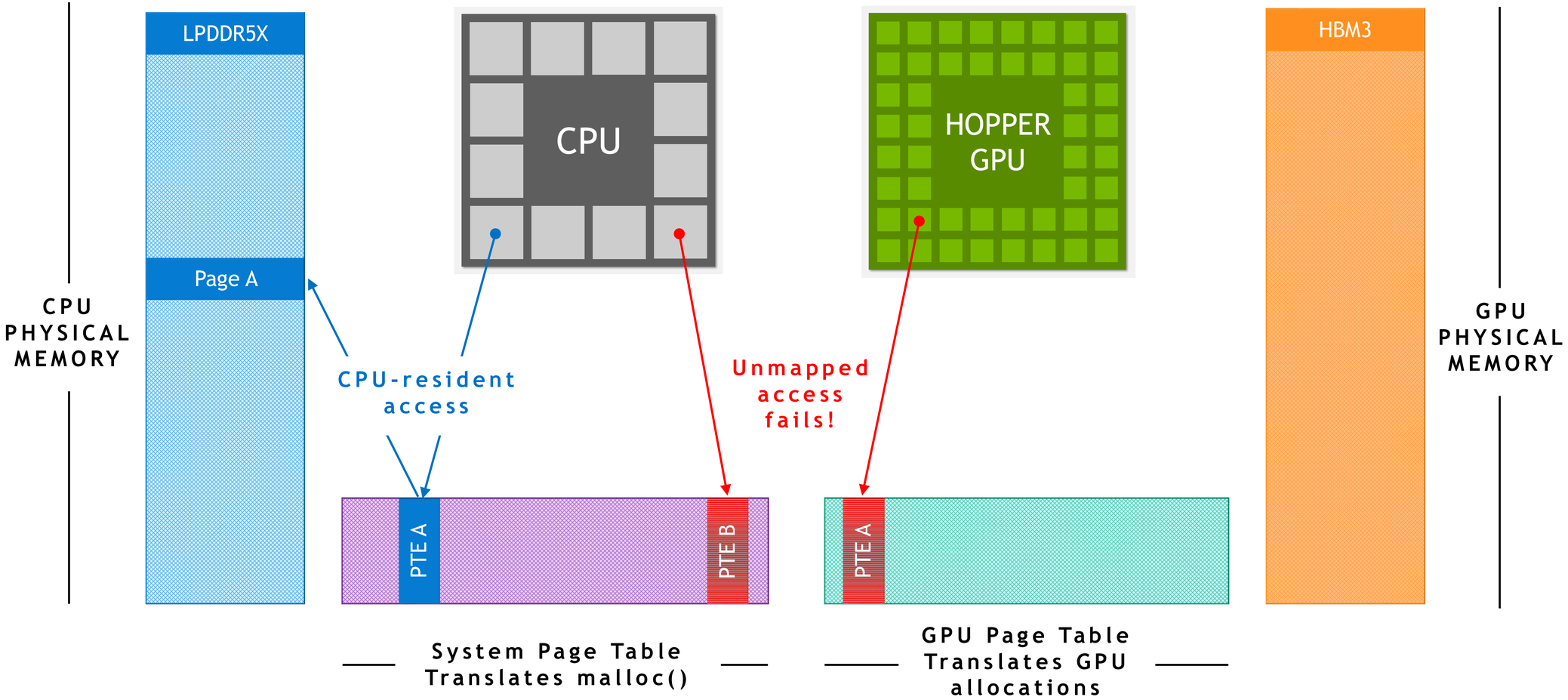
### 2023

**HMM**

CUDA 12.2u2: Heterogeneous Memory Management released in CUDA

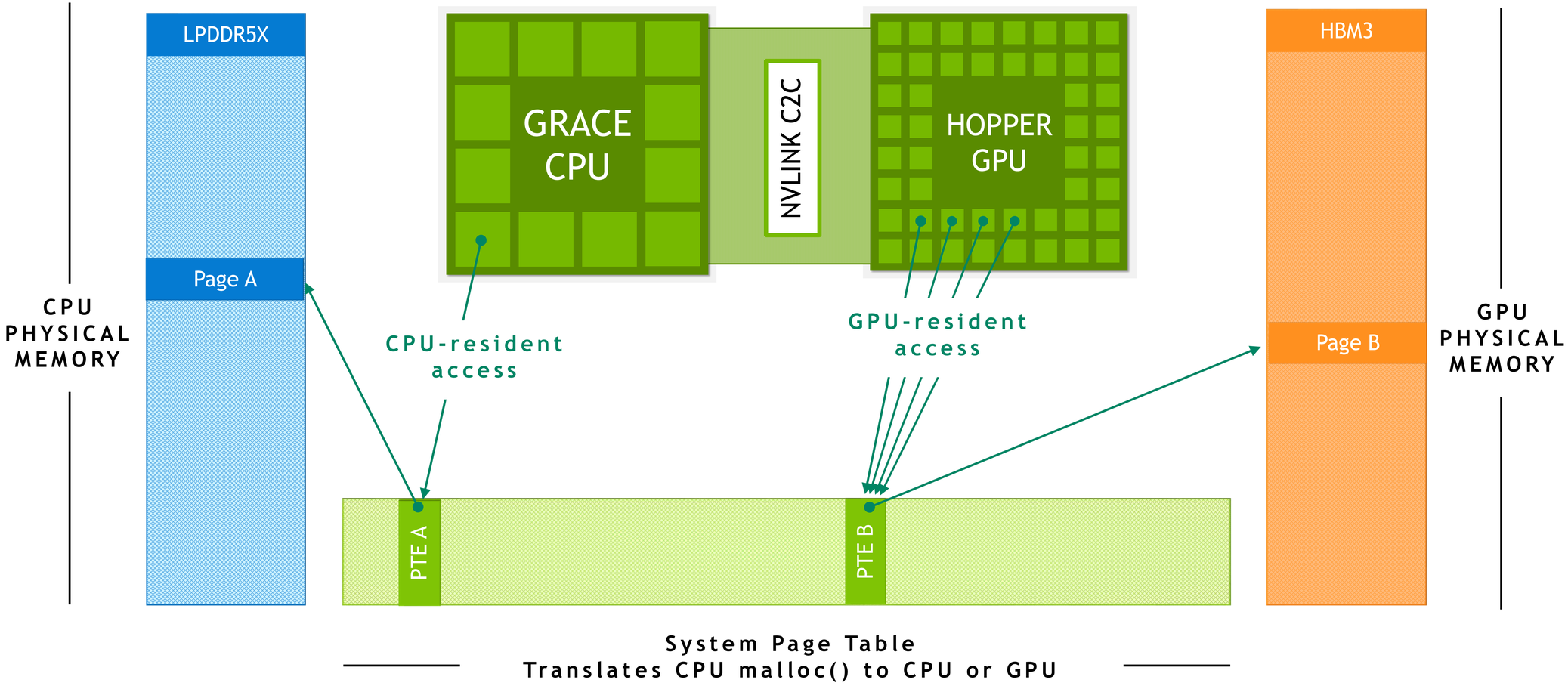
# (RECAP) HW/SW MEMORY VIEW ON X86 + GPU

Separate page tables



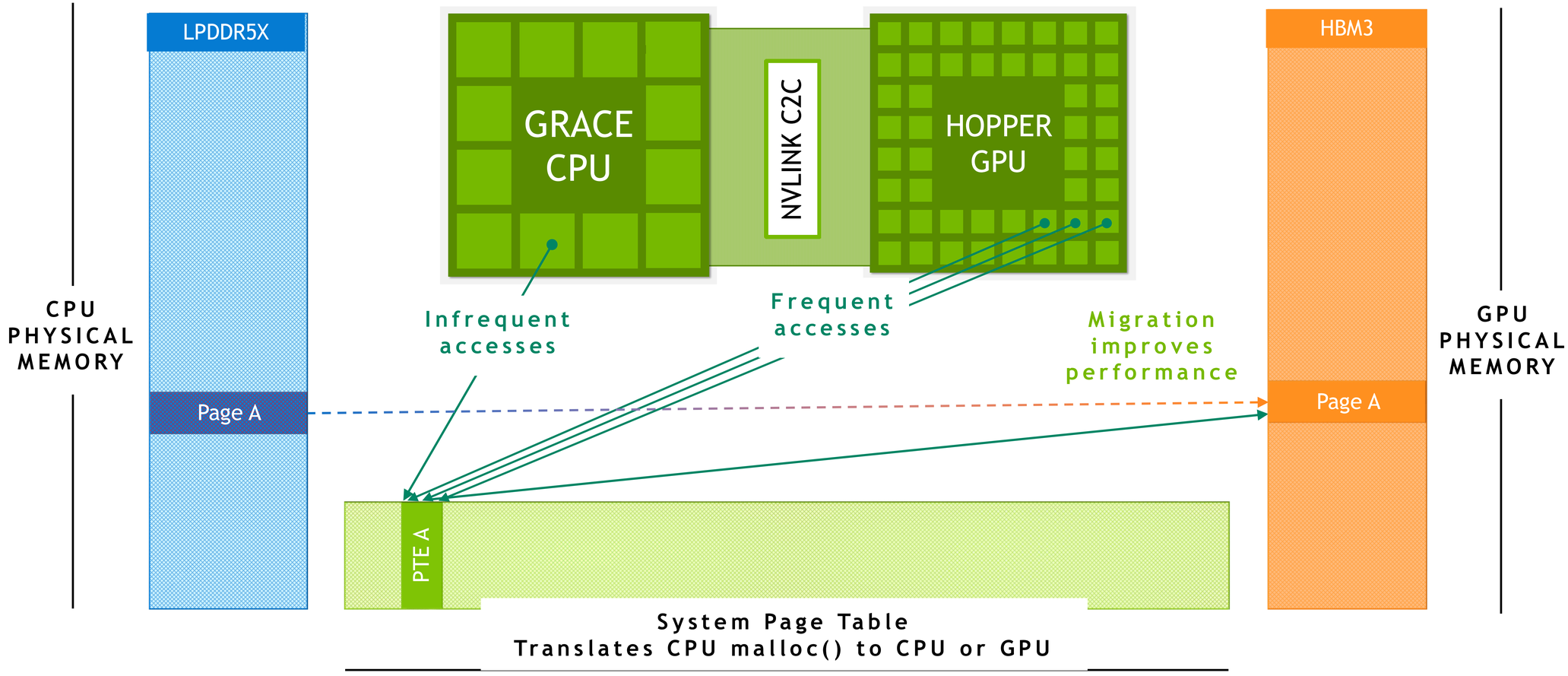
# HW/SW memory view on GH200

Local memory pool accesses at link speeds



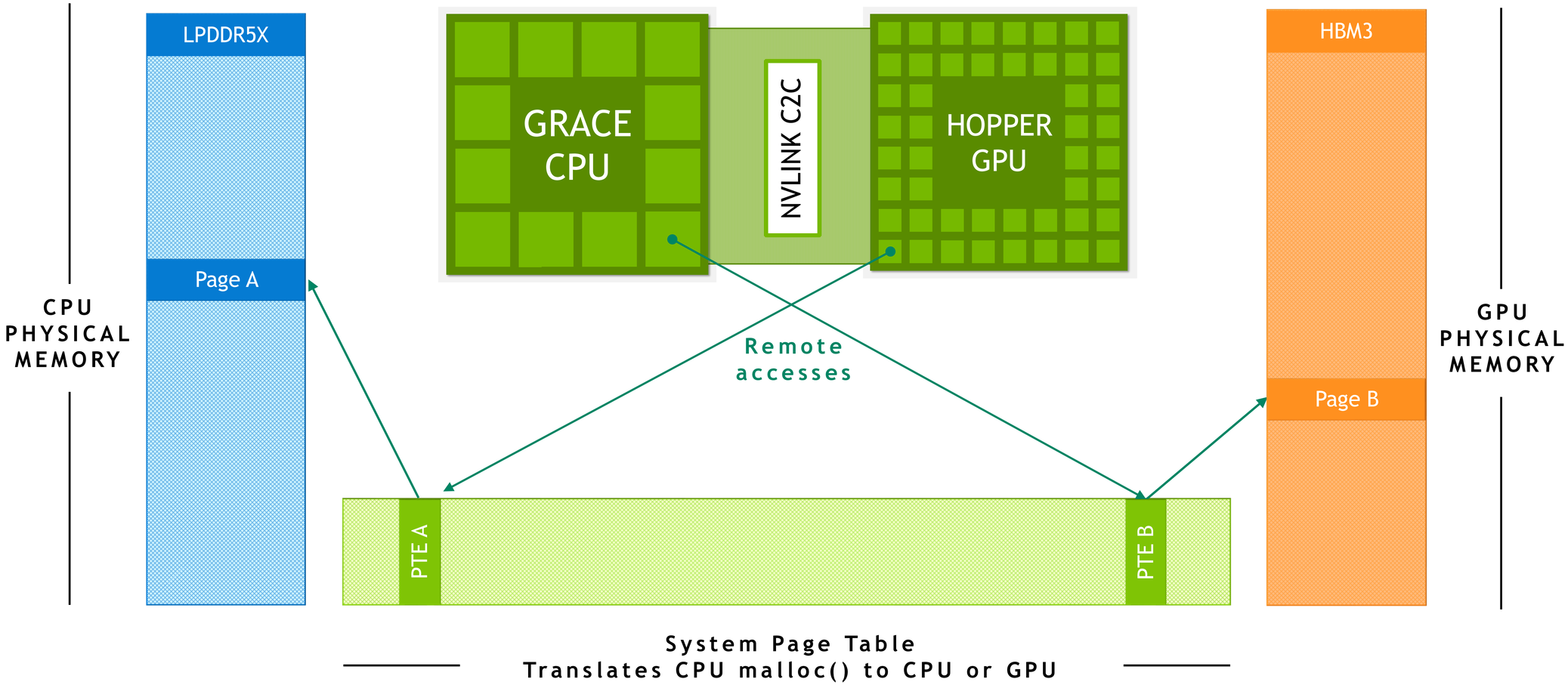
# HW/SW MEMORY VIEW ON GH200

Simplified Unified Memory via **Address Translation Service (ATS)** & automatic micrations



# HW/SW MEMORY VIEW ON GH200

C2C enabling remote accesses



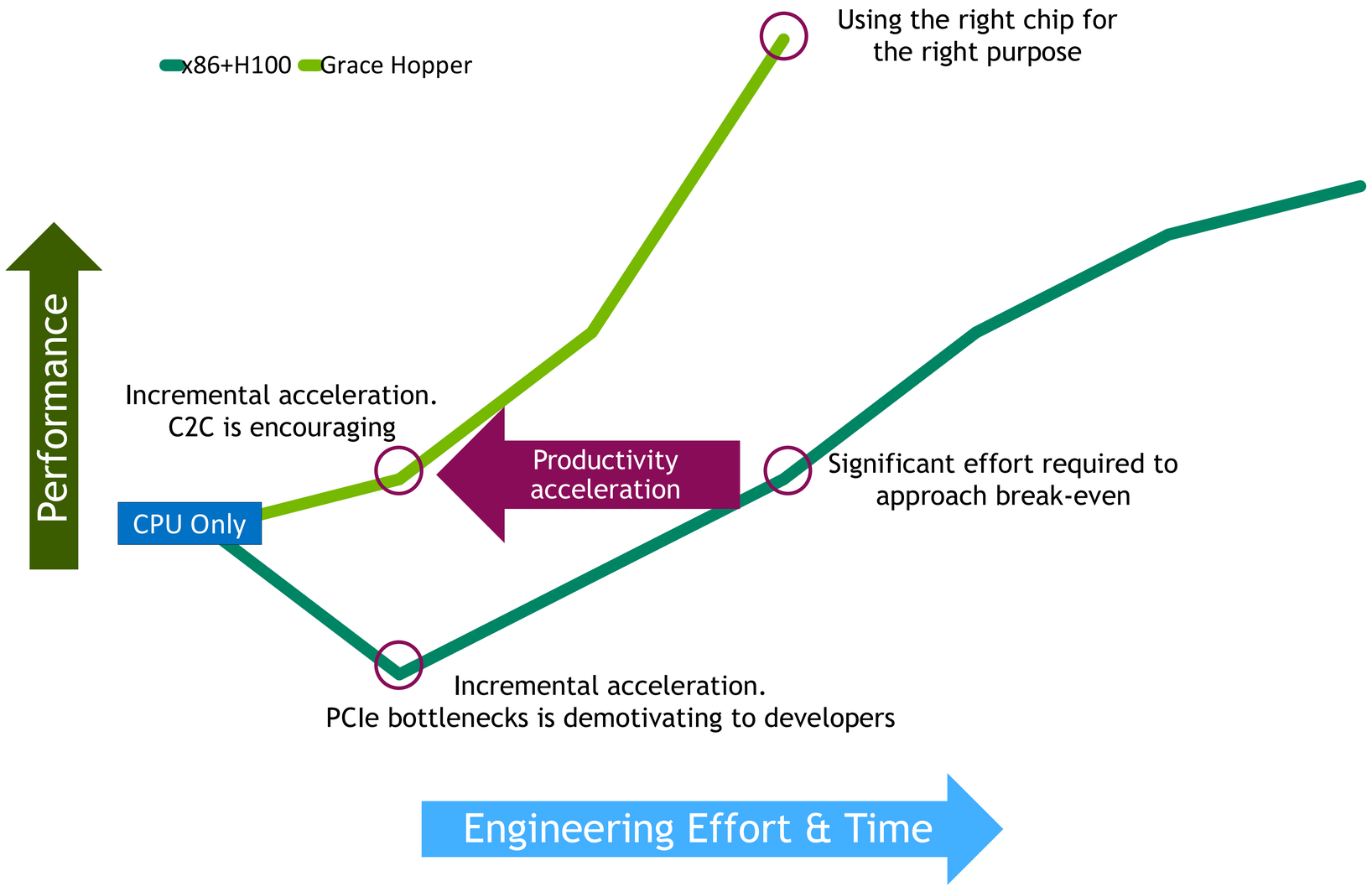
# What do Programmers Need to Know About Grace-Hopper

- **Existing GPU applications require no changes for Grace Hopper**
  - No new APIs
  - No restructuring
  - No new programming model
  - Developers who choose to can optimize for the Grace Hopper platform
- **Existing GPU applications (fully or partially ported) will run better on Grace Hopper**
  - Data migration no longer required, may still be a performance optimization
  - When data migrations happen, they happen faster due to C2C interconnect
  - CPU code will benefit from higher bandwidth memory, high thread performance, coherent accesses
  - Existing, stable Unified Memory APIs may be used for performance optimization
- **Non-GPU applications will run unmodified and benefit from Grace architecture**
- **Porting from CPU to GPU is made simpler by Grace Hopper**
  - Coherent Memory Subsystem
  - C2C interconnect
  - Programming model choice
- **Some new capabilities may be unlocked**
  - Managing and processing larger data sets
  - Workflows that utilize both halves



# Developer Journey with Grace Hopper

Accelerating the path to accelerated computing

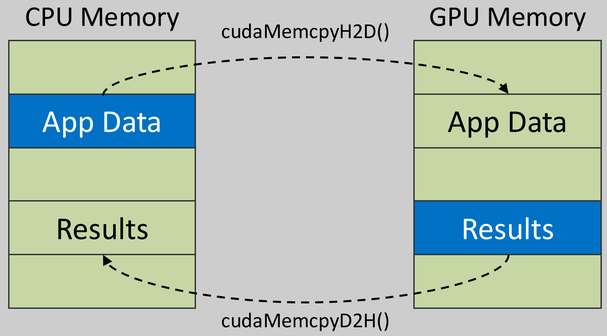


# The Grace Hopper Advantage

Full CUDA support with additional Grace memory extensions

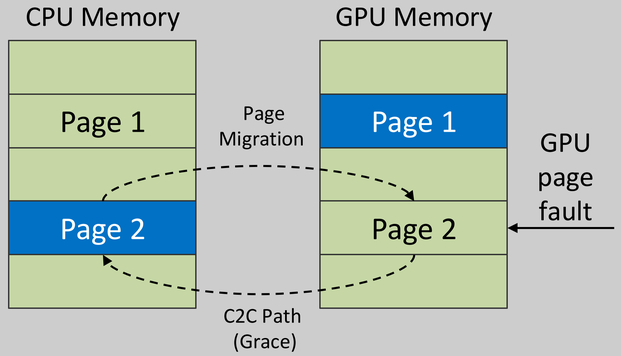
## Explicit Copy

Application explicitly moves data between CPU & GPU as needed



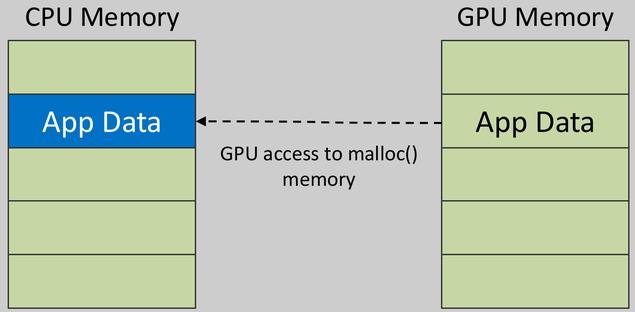

## Managed Memory

CPU and GPU can access memory on-demand and data migrated locally for higher BW access




## System Allocated

GPU can access memory allocated from `malloc()`, `mmap()`, etc.



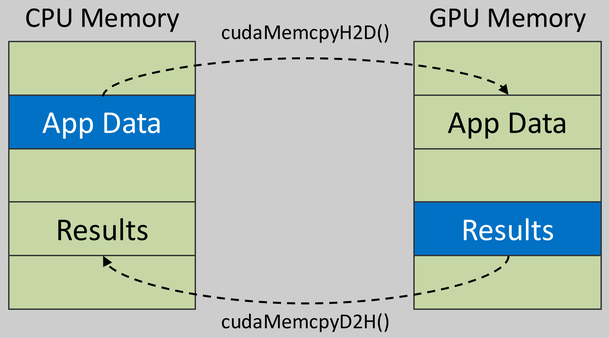

| | | | |
|---|---|---|---|
| **HGX** | ~60 GB/s PCIe Gen5 transfers (H2D/D2H) | Requires migration to GPU | Access possible with explicit call to `cudaHostRegister()` at PCIe speeds<br>Requires HMM patch in Linux Kernel |
| **G+H** | 7x faster transfers, up to 450 GB/s (NVLink C2C) | Migrations not required and faster migrations when they happen at NVLink C2C speed | `cudaHostRegister()` not needed; access at NVLink C2C speeds |

# The Grace Hopper Advantage

Full CUDA support with additional Grace memory extensions

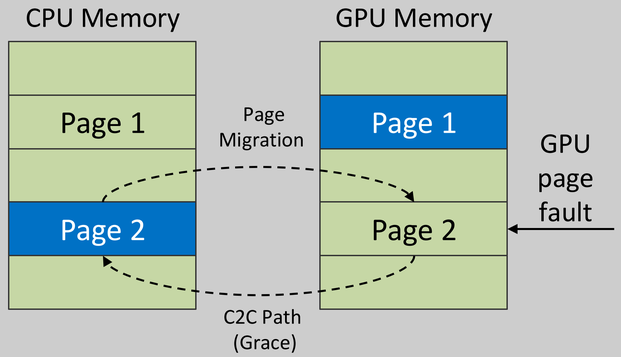
## Explicit Copy

Application explicitly moves data between CPU & GPU as needed



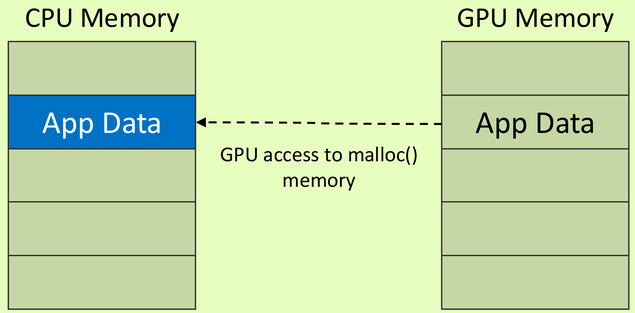

## Managed Memory

CPU and GPU can access memory on-demand and data migrated locally for higher BW access




## System Allocated

GPU can access memory allocated from `malloc()`, `mmap()`, etc.




| | | | |
|---|---|---|---|
| **HGX** | ~60 GB/s PCIe Gen5 transfers (H2D/D2H) | Requires migration to GPU | Access possible with explicit call to `cudaHostRegister()` at PCIe speeds<br>Requires HMM patch in Linux Kernel |
| **G+H** | 7x faster transfers, up to 450 GB/s (NVLink C2C) | Migrations not required and faster migrations when they happen at NVLink C2C speed | `cudaHostRegister()` not needed; access at NVLink C2C speeds |

# Using System Allocator using CUDA

Super easy to move codes to GPU, portable with HMM on x86

```
__global__ void kernel(int *data)
{
    data[threadIdx.x] += 2;
}

int main()
{
  int N = 128;
  int *data = (int *) malloc(N * sizeof(int));

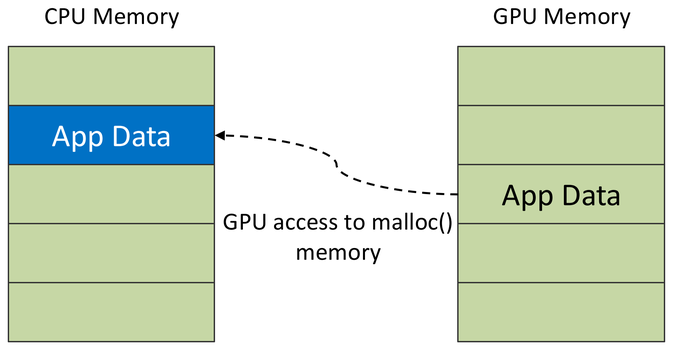
  for (int i = 0; i < N; i++)
  { data[i] = i; }

  kernel<<<1, N>>>(data);
  cudaDeviceSynchronize();

  for (int i = 0; i < 128; i++)
  { printf("%d ", data[i]); }

  free(data);
}
```

- **On x86**: enabled by HMM, data migrates based on page fault mechanism
- **On Grace-Hopper**:
  - GPU can access memory allocated from `malloc()`, `mmap()`, etc.
    - Both stack and heap are accessible
  - Translation done via ATS
  - Direct load/store can be performed by both CPU and GPU. No fault-based migration.
  - Fine grained synchronization can be done using atomics
  - Data can migrate based on user hints or driver heuristics.




**PCIE:** Access possible with explicit call to `cudaHostRegister()` at PCIe speeds

**Grace:** `cudaHostRegister()` not needed; access at NVLink C2C speeds

# CUDA Unified/Managed Memory

Portable codes across x86 and Grace Hopper by moving memory pages

```
__global__ void kernel(int *data)
{
    data[threadIdx.x] += 2;
}

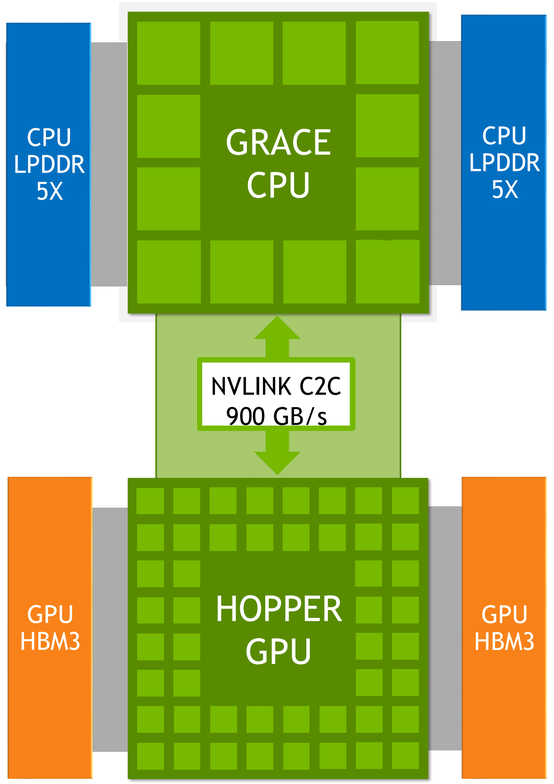
int main()
{
    int N = 128;

    int *data;
    cudaMallocManaged((void **)&data, N * sizeof(int));

    for (int i = 0; i < N; i++)
    {
        data[i] = i;
    }

    kernel<<<1, N>>>(data);
    cudaDeviceSynchronize();

    cudaFree(data);
}
```



# CUDA Unified/Managed Memory

Portable codes across x86 and Grace Hopper by moving memory pages

```
__global__ void kernel(int *data)
{
    data[threadIdx.x] += 2;
}

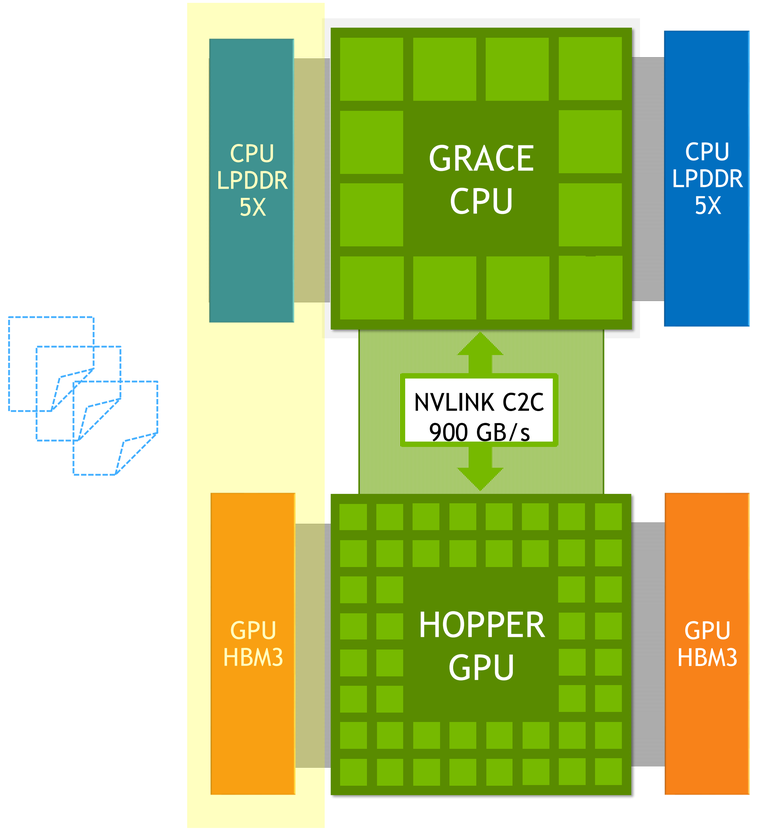
int main()
{
    int N = 128;

    int *data;
    cudaMallocManaged((void **)&data, N * sizeof(int));

    for (int i = 0; i < N; i++)
    {
        data[i] = i;
    }

    kernel<<<1, N>>>(data);
    cudaDeviceSynchronize();

    cudaFree(data);
}
```



# CUDA Unified/Managed Memory

Portable codes across x86 and Grace Hopper by moving memory pages

```
__global__ void kernel(int *data)
{
    data[threadIdx.x] += 2;
}

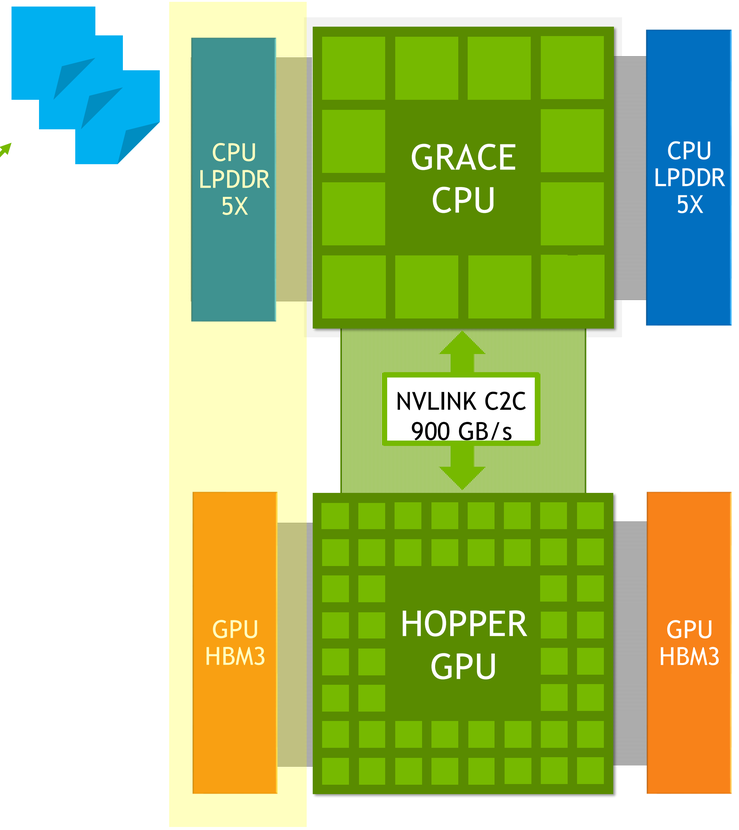
int main()
{
    int N = 128;

    int *data;
    cudaMallocManaged((void **)&data, N * sizeof(int));

    for (int i = 0; i < N; i++)
    {
        data[i] = i;
    }

    kernel<<<1, N>>>(data);
    cudaDeviceSynchronize();

    cudaFree(data);
}
```



# CUDA Unified/Managed Memory

Portable codes across x86 and Grace Hopper by direct access

```
__global__ void kernel(int *data)
{
    data[threadIdx.x] += 2;
}

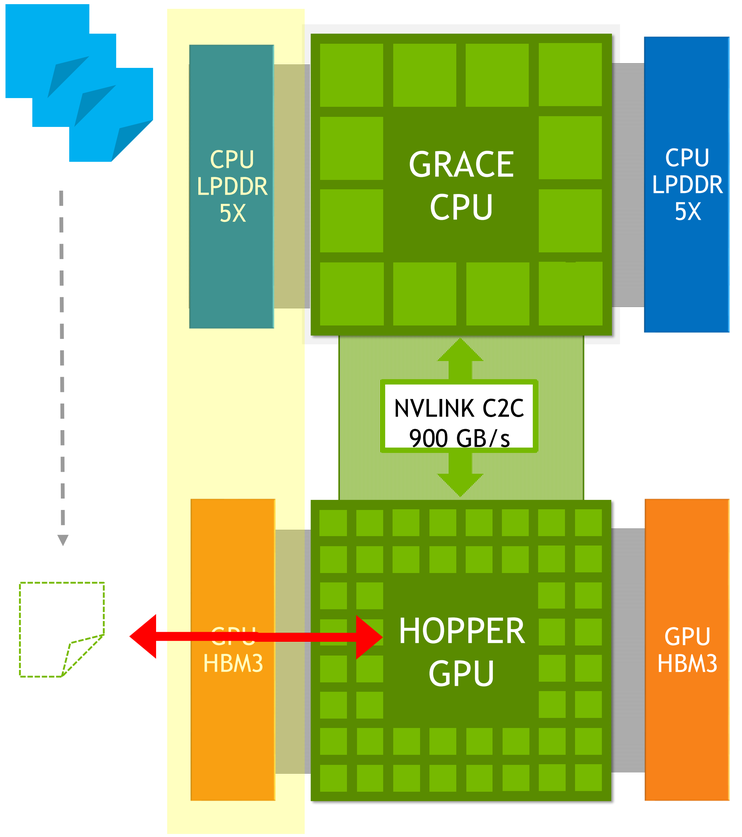
int main()
{
    int N = 128;

    int *data;
    cudaMallocManaged((void **)&data, N * sizeof(int));

    for (int i = 0; i < N; i++)
    {
        data[i] = i;
    }

    kernel<<<1, N>>>(data);
    cudaDeviceSynchronize();

    cudaFree(data);
}
```




NO DIFFERENCE THAN x86+GPU BEHAVIOUR, JUST BETTER

# CUDA Unified/Managed Memory

Portable codes across x86 and Grace Hopper by moving memory pages

```
__global__ void kernel(int *data)
{
    data[threadIdx.x] += 2;
}

int main()
{
    int N = 128;

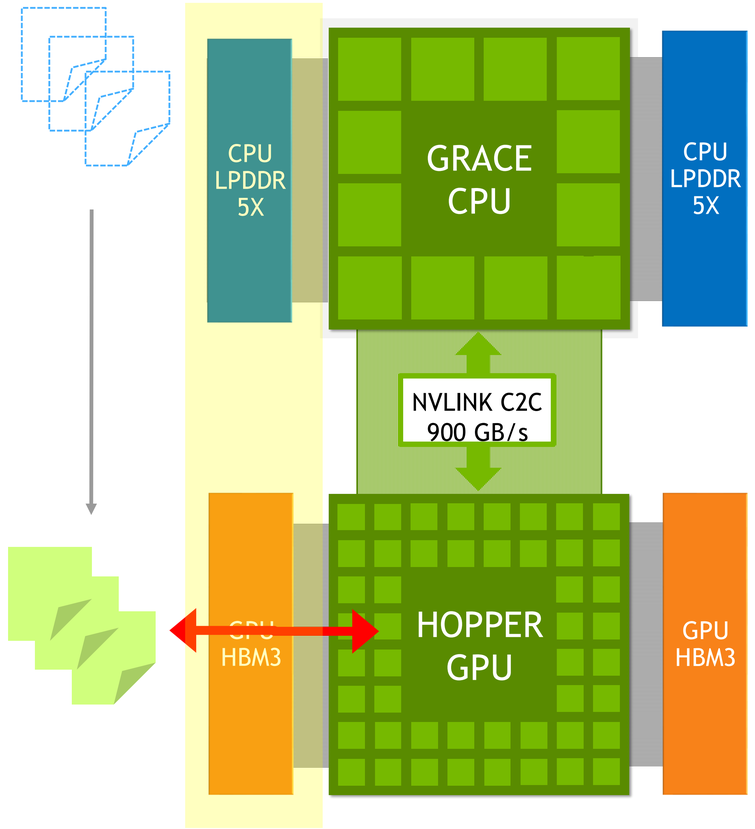
    int *data;
    cudaMallocManaged((void **)&data, N * sizeof(int));

    for (int i = 0; i < N; i++)
    {
        data[i] = i;
    }

    cudaMemPrefetchAsync(data, N * sizeof(int), 0);

    kernel<<<1, N>>>(data);
    cudaDeviceSynchronize();

    cudaFree(data);
}
```




NO DIFFERENCE THAN x86+GPU BEHAVIOUR, JUST BETTER

# System Allocator

Easy to move codes to GPU. Uses Address Translation Service (ATS)

```
__global__ void kernel(int *data)
{
    data[threadIdx.x] += 2;
}

int main()
{
  int N = 128;

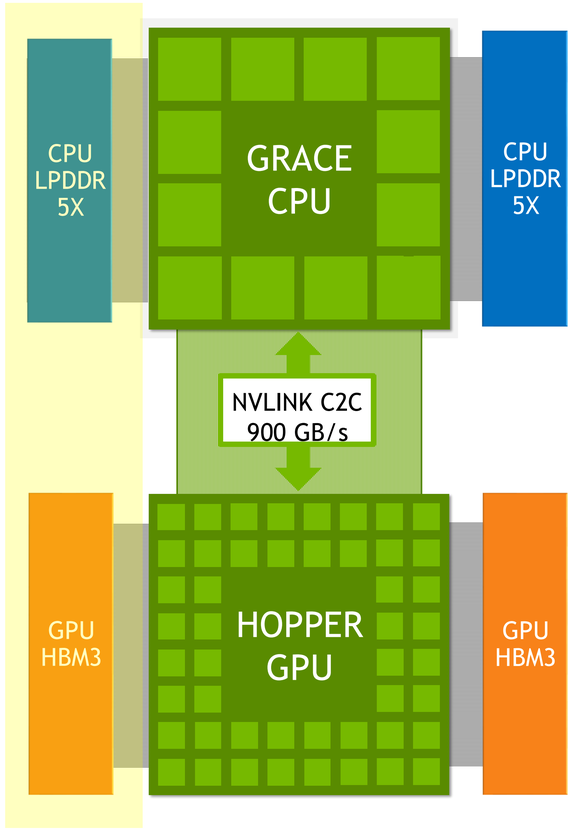
  int *data = (int *) malloc(N * sizeof(int));

  for (int i = 0; i < N; i++)
  { data[i] = i; }

  kernel<<<1, N>>>(data);
  cudaDeviceSynchronize();

  for (int i = 0; i < 128; i++)
  { printf("%d ", data[i]); }

  free(data);
}
```



# System Allocator

Easy to move codes to GPU. Uses Address Translation Service (ATS)

```
__global__ void kernel(int *data)
{
    data[threadIdx.x] += 2;
}

int main()
{
  int N = 128;

  int *data = (int *) malloc(N * sizeof(int));

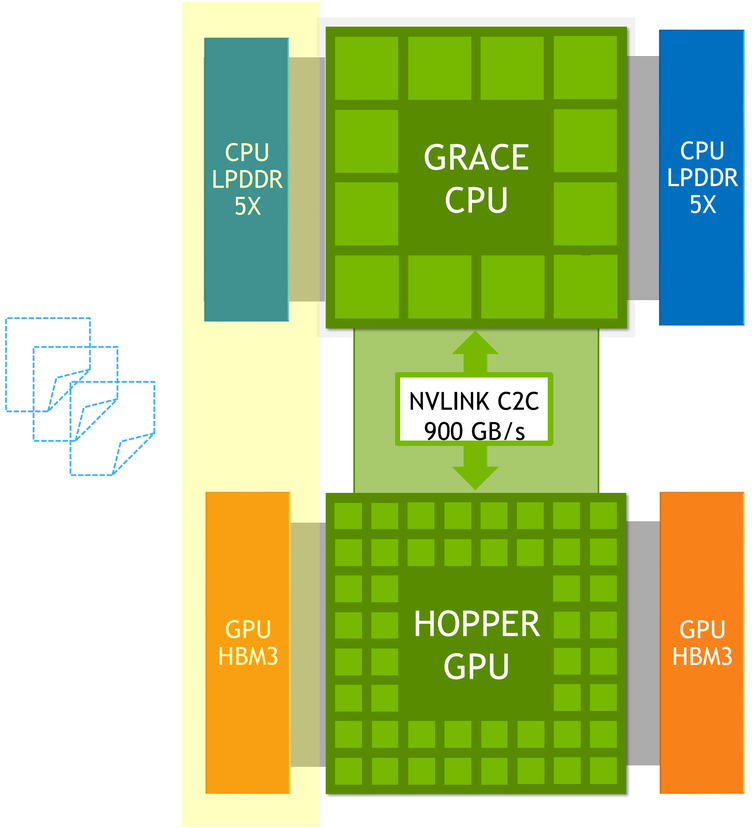
  for (int i = 0; i < N; i++)
  { data[i] = i; }

  kernel<<<1, N>>>(data);
  cudaDeviceSynchronize();

  for (int i = 0; i < 128; i++)
  { printf("%d ", data[i]); }

  free(data);
}
```

Physical pages are not yet allocated



# System Allocator

Easy to move codes to GPU. Uses Address Translation Service (ATS)

```
__global__ void kernel(int *data)
{
    data[threadIdx.x] += 2;
}

int main()
{
  int N = 128;

  int *data = (int *) malloc(N * sizeof(int));

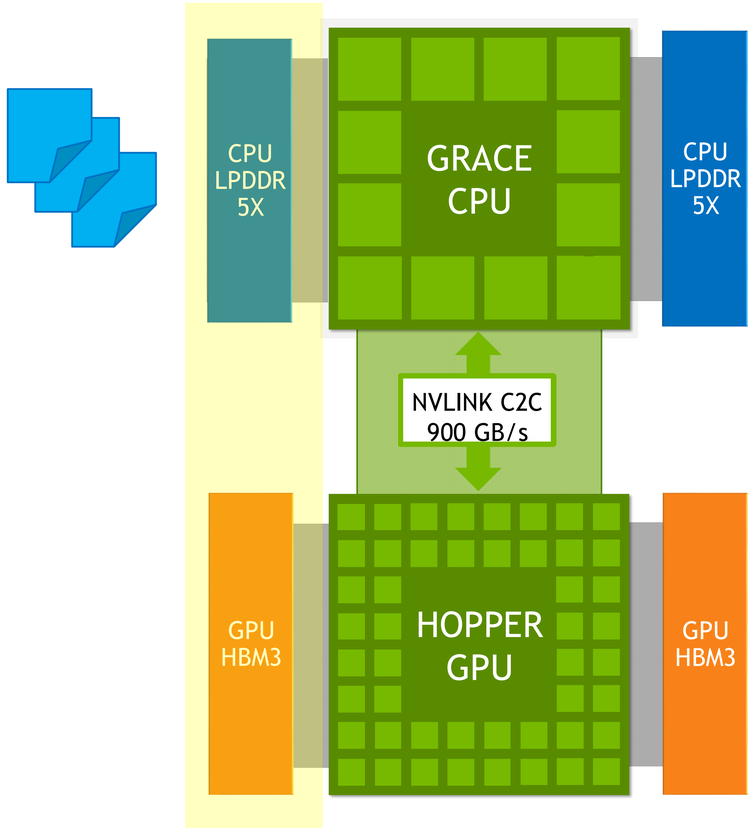
  for (int i = 0; i < N; i++)
  { data[i] = i; }

  kernel<<<1, N>>>(data);
  cudaDeviceSynchronize();

  for (int i = 0; i < 128; i++)
  { printf("%d ", data[i]); }

  free(data);
}
```

Physical pages are allocated on first touch



# System Allocator

Easy to move codes to GPU. Uses Address Translation Service (ATS)

```
__global__ void kernel(int *data)
{
    data[threadIdx.x] += 2;
}

int main()
{
  int N = 128;

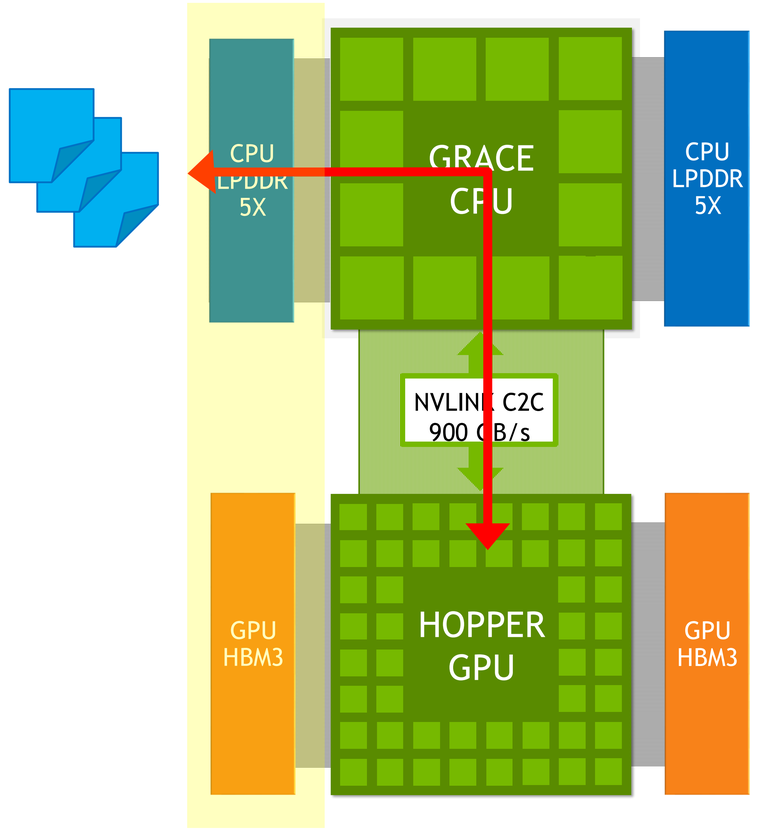
  int *data = (int *) malloc(N * sizeof(int));

  for (int i = 0; i < N; i++)
  { data[i] = i; }

  kernel<<<1, N>>>(data);
  cudaDeviceSynchronize();

  for (int i = 0; i < 128; i++)
  { printf("%d ", data[i]); }

  free(data);
}
```



# System Allocator

Easy to move codes to GPU. Uses Address Translation Service (ATS)

```
__global__ void kernel(int *data)
{
    data[threadIdx.x] += 2;
}

int main()
{
  int N = 128;

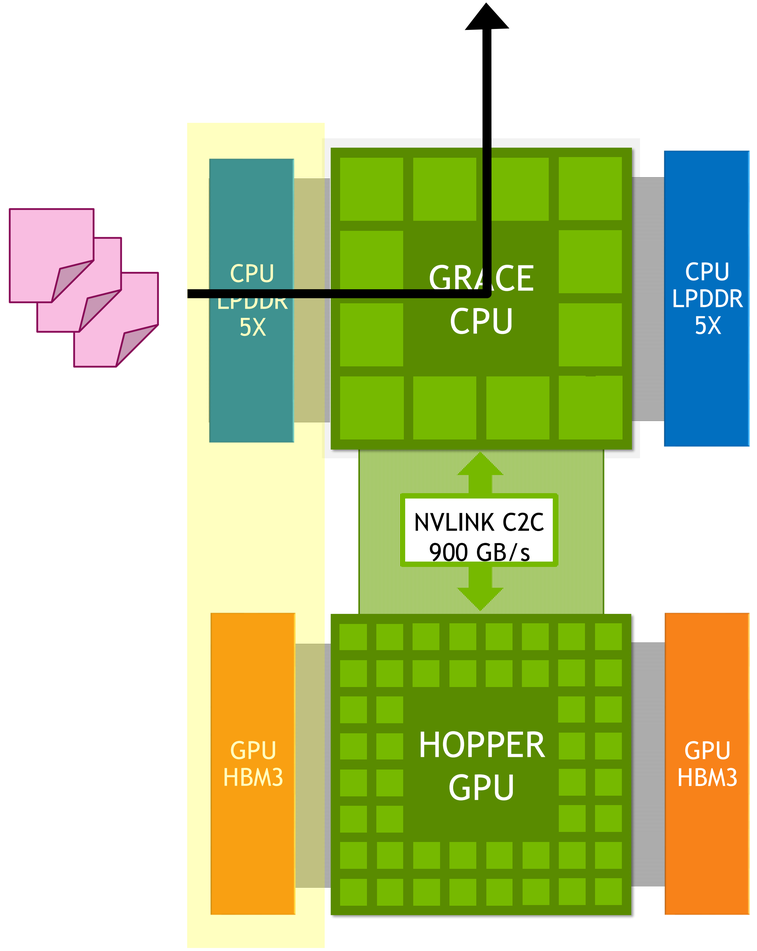
  int *data = (int *) malloc(N * sizeof(int));

  for (int i = 0; i < N; i++)
  { data[i] = i; }

  kernel<<<1, N>>>(data);
  cudaDeviceSynchronize();

  for (int i = 0; i < 128; i++)
  { printf("%d ", data[i]); }

  free(data);
}
```



# System Allocator

Easy to move codes to GPU. Uses Address Translation Service (ATS)

```
__global__ void kernel(int *data1, int *data2)
{
  data1[threadIdx.x] = threadIdx.x;
  data2[threadIdx.x] = threadIdx.x;
}

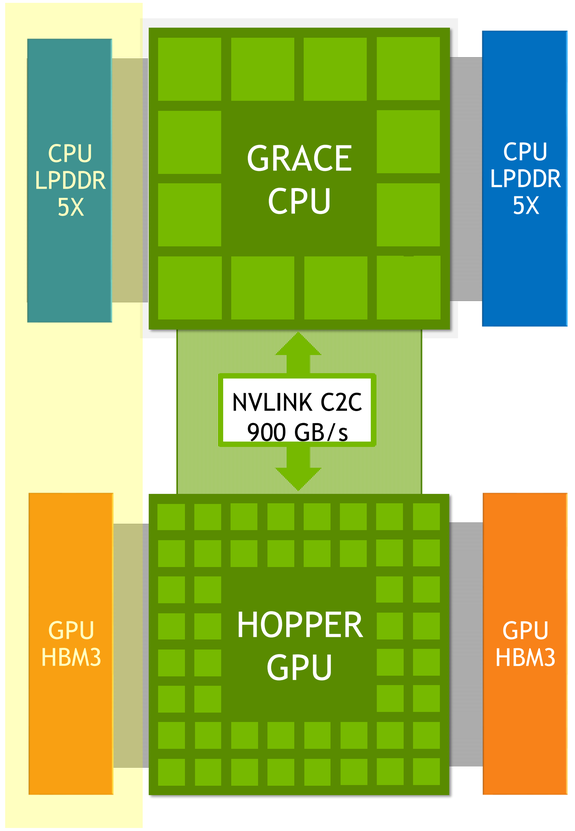
int main()
{
  int *data = (int *) malloc (128 * sizeof(int));

  int data2[128];

  kernel<<<1, 128>>>(data, data2);
  cudaDeviceSynchronize();

  for (int i = 0; i < 128; i++)
  {
    printf("(%d, %d)  ", data[i], data2[i]);
  }

  free(data);
  return 0;
}
```



# System Allocator

Easy to move codes to GPU. Uses Address Translation Service (ATS)

```
__global__ void kernel(int *data1, int *data2)
{
  data1[threadIdx.x] = threadIdx.x;
  data2[threadIdx.x] = threadIdx.x;
}

int main()
{
  int *data = (int *) malloc (128 * sizeof(int));

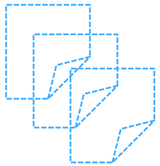
  int data2[128];

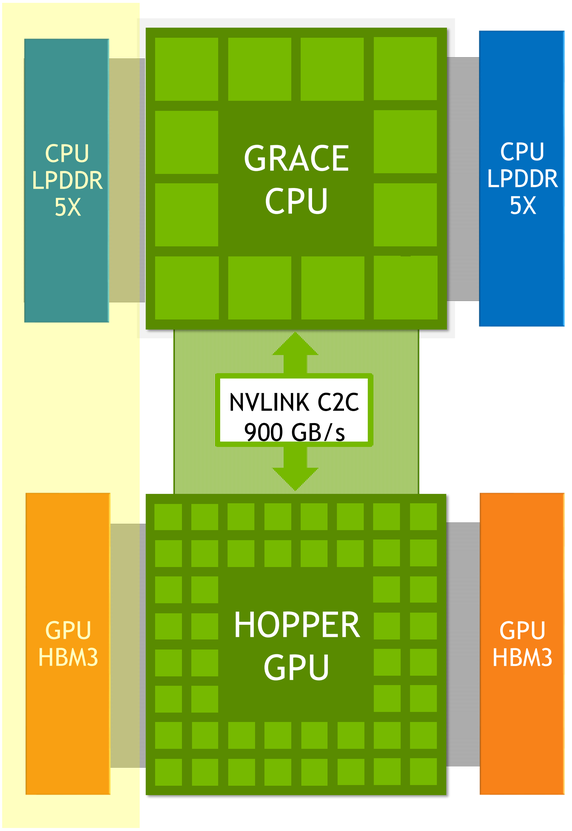
  kernel<<<1, 128>>>(data, data2);
  cudaDeviceSynchronize();

  for (int i = 0; i < 128; i++)
  {
    printf("(%d, %d)  ", data[i], data2[i]);
  }

  free(data);
  return 0;
}
```

Dynamic allocation, physical pages are not yet allocated





# System Allocator

Easy to move codes to GPU. Uses Address Translation Service (ATS)

```
__global__ void kernel(int *data1, int *data2)
{
  data1[threadIdx.x] = threadIdx.x;
  data2[threadIdx.x] = threadIdx.x;
}

int main()
{
  int *data = (int *) malloc (128 * sizeof(int));

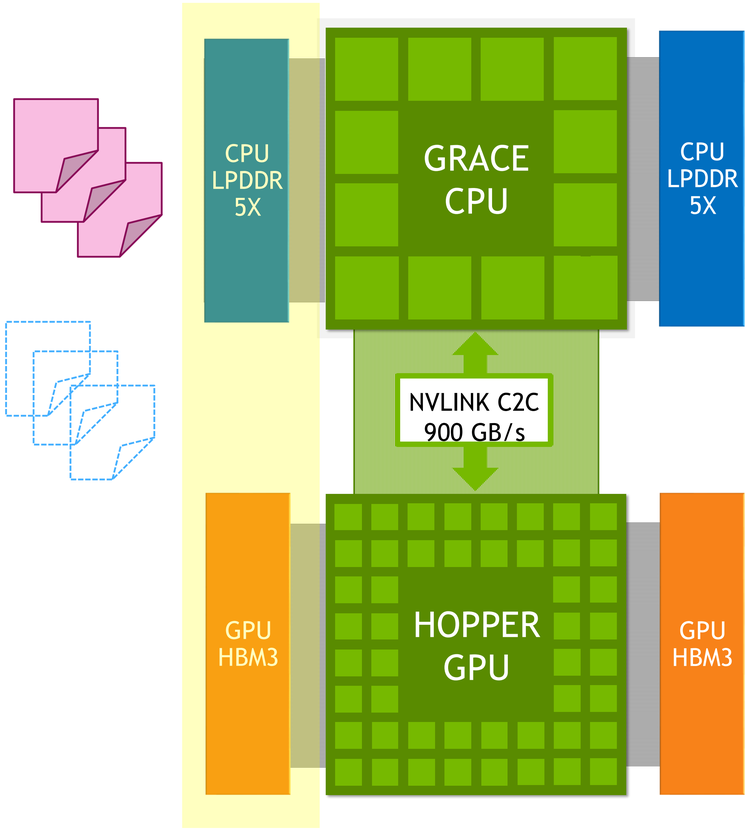
  int data2[128];

  kernel<<<1, 128>>>(data, data2);
  cudaDeviceSynchronize();

  for (int i = 0; i < 128; i++)
  {
    printf("(%d, %d)  ", data[i], data2[i]);
  }

  free(data);
  return 0;
}
```

Static Allocation on Heap



# System Allocator

Easy to move codes to GPU. Uses Address Translation Service (ATS)

```
__global__ void kernel(int *data1, int *data2)
{
  data1[threadIdx.x] = threadIdx.x;
  data2[threadIdx.x] = threadIdx.x;
}

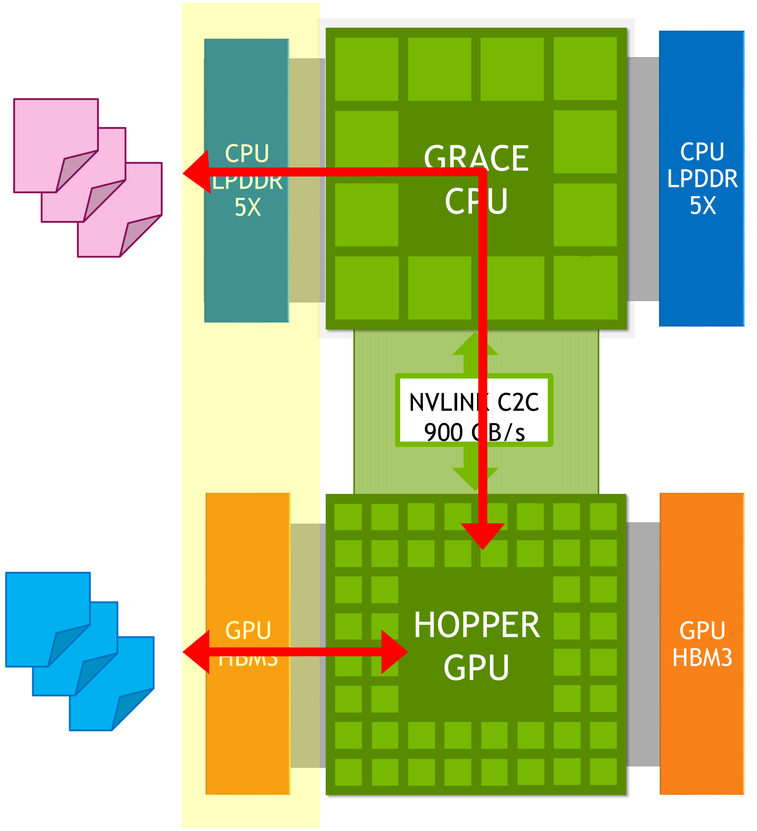
int main()
{
  int *data = (int *) malloc (128 * sizeof(int));

  int data2[128];

  kernel<<<1, 128>>>(data, data2);
  cudaDeviceSynchronize();

  for (int i = 0; i < 128; i++)
  {
    printf("(%d, %d)  ", data[i], data2[i]);
  }

  free(data);
  return 0;
}
```



# System Allocator

Easy to move codes to GPU. Uses Address Translation Service (ATS)

```
__global__ void kernel(int *data1, int *data2)
{
  data1[threadIdx.x] = threadIdx.x;
  data2[threadIdx.x] = threadIdx.x;
}

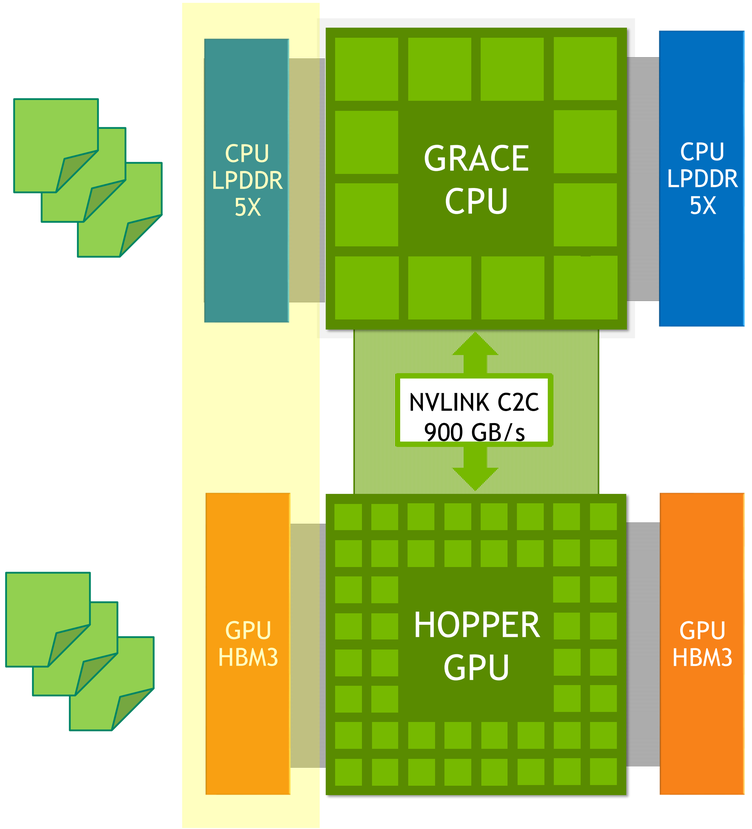
int main()
{
  int *data = (int *) malloc (128 * sizeof(int));

  int data2[128];

  kernel<<<1, 128>>>(data, data2);
  cudaDeviceSynchronize();

  for (int i = 0; i < 128; i++)
  {
    printf("(%d, %d)  ", data[i], data2[i]);
  }

  free(data);
  return 0;
}
```



# System Allocator

Easy to move codes to GPU. Uses Address Translation Service (ATS)

```
__global__ void kernel(int *data1, int *data2)
{
  data1[threadIdx.x] = threadIdx.x;
  data2[threadIdx.x] = threadIdx.x;
}

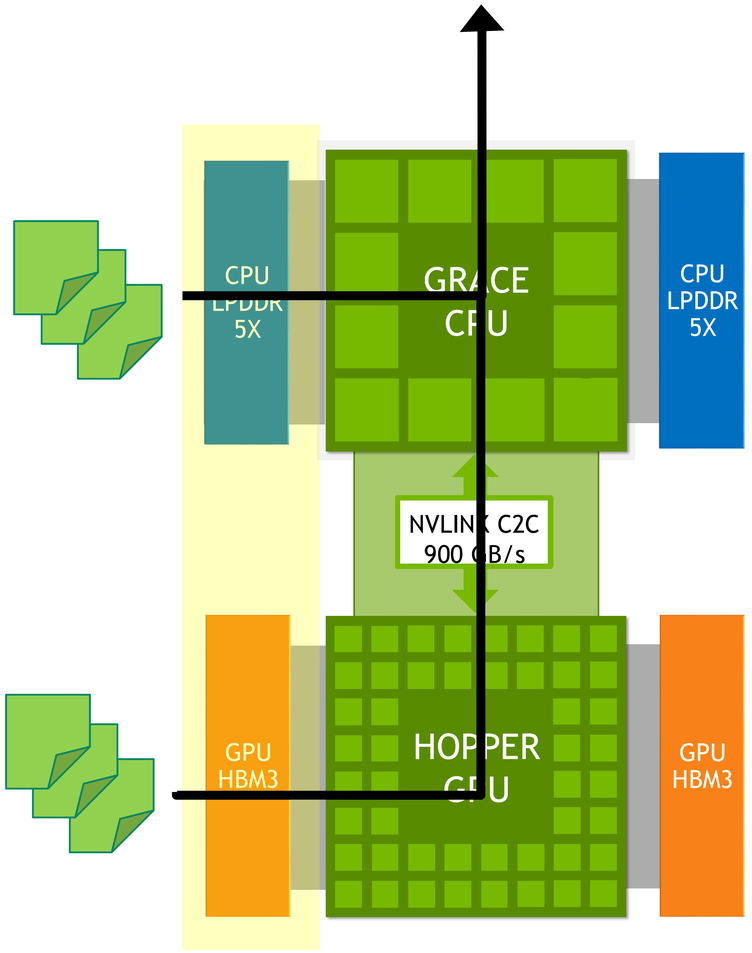
int main()
{
  int *data = (int *) malloc (128 * sizeof(int));

  int data2[128];

  kernel<<<1, 128>>>(data, data2);
  cudaDeviceSynchronize();

  for (int i = 0; i < 128; i++)
  {
    printf("(%d, %d)  ", data[i], data2[i]);
  }

  free(data);
  return 0;
}
```



# CUDA Explicit Memory Allocators

Maximum **portable** performance out-of-the-box

- **No programming model changes!**
  → No new APIs
  → No changes to existing APIs
  → No source code changes

- **Unified Memory**
  - Available on *most* platforms supported by CUDA 12.x:
    GH, P9+V100, PCIe x86 & Arm, etc.
  - Same Unified Memory Programming Model for all platforms
    - *"memory accesses just work" + "hints".*

- Unified Memory **Hints**
  - *Hints* only impact performance, not results.
  - **cudaMemAdvise** hints: PreferredLocation, AccessedBy.
  - **cudaMemPrefetch** hints: prefetch to NUMA node.
  - Work with all memory, e.g., including `malloc`.

| Memory | Placement | Access-based Migration | Accessible From | |
|---|---|---|---|---|
| | | | CPU | GPUs |
| **System-allocated** (malloc, mmap, new) | First-touch (GPU \| CPU) | ✅ | ✅ | ✅ |
| **CUDA managed** (cudaMallocManaged) | | ✅ | ✅ | ✅ |
| **CUDA device memory** (cudaMalloc) | GPU | ❌ | ❌ | ✅ |
| **CUDA host memory** (cudaMallocHost) | CPU | ❌ | ✅ | ✅ |
| *…and many others: interprocess, virtual, fabric, …* | | | | |

## CUDA Unified Memory Hints

```
cudaMemAdvise(ptr, nbytes, advice, device);
```

| Advices | PreferredLocation | AccessedBy | ReadMostly |
|---|---|---|---|

| Devices | GPU id | CPU |
|---|---|---|

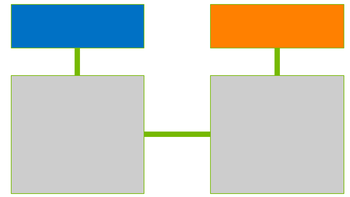
```
cudaMemPrefetchASync(ptr, nbytes, destination);
```

| Destinations | GPU id | CPU |
|---|---|---|

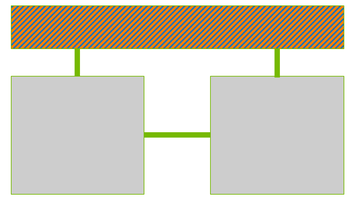
# NVHPC Compilers GPU Memory Model

Abstraction over HW simplifying GPU programming

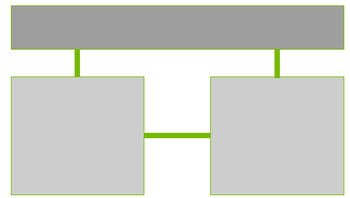
- **Separate (-gpu=mem:separate)** - CPU and GPU have distinct memories when data are shared between the two explicit copy is needed



- **Managed (-gpu=mem:managed)** – CPU and GPU have a single address space for dynamically-allocated data, data is migrated automatically on-demand. Other memory remains separate.
  - Only dynamically allocated data are shared between CPU and GPU
  - Stack and global variables outside of parallel algorithm code can't be accessed in parallel algorithm
  - Binaries compiled for such mode can run on systems with CUDA Managed Memory only or full CUDA Unified Memory support



- **Unified (-gpu=mem:unified)** - CPU and GPU have single address space which allows accessing all data locations from both processors, data may be migrated or accessed in-place.
  - All CPU data are accessible from the GPU
  - Utilizing full CUDA Unified Memory (with ATS/HMM)
  - Binaries compiled for such mode can run on systems with full CUDA Unified Memory support



# How to use UVM in PyTorch?

## Training AI use-case

**Assumption**: I have only 1 GPU available

**User scenario**: loading a model (e.g. Llama 3 70B) bigger than the physical memory on the GPU

Error message:

OutOfMemoryError: CUDA out of memory. Tried to allocate 896.00 MiB. GPU 0 has a total capacity of 95.00 GiB of which 524.06 MiB is free. Including non-PyTorch memory, this process has 86.45 GiB memory in use.

```
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 570.124.06             Driver Version: 570.124.06     CUDA Version: 12.8     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA GH200 480GB             Off |   00000009:01:00.0 Off |                    0 |
| N/A   32C    P0            111W /  900W |   96746MiB /  97871MiB |      0%      Default |
|                                         |                        |             Disabled |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|    0   N/A  N/A           1207      C   /usr/bin/python                       88524MiB |
+-----------------------------------------------------------------------------------------+
```

# How to use UVM in PyTorch?

## OPTION 1
### torch.cuda.memory.CUDAPluggableAllocator

```
// filename alloc.cc
#include <cuda_runtime_api.h>
#include <iostream>

extern "C" {
void* my_malloc(ssize_t size, int device, cudaStream_t stream) {
  void *ptr;
  cudaMallocManaged(&ptr, size);
  std::cout<<"alloc "<<ptr<<size<<std::endl;
  return ptr;
}

void my_free(void* ptr, ssize_t size, int device, cudaStream_t stream) {
  std::cout<<"free "<<ptr<< " "<<stream<<std::endl;
  cudaFree(ptr);
}
}
```

```
import torch

# Load the allocator
new_alloc = torch.cuda.memory.CUDAPluggableAllocator(
   'alloc.so', 'my_malloc', 'my_free')

# Swap the current allocator
torch.cuda.memory.change_current_allocator(new_alloc)
```

## OPTION 2
### Using Rapids Memory Manager (RMM) module

```
import torch
import rmm
from rmm.allocators.torch import rmm_torch_allocator

# …

rmm.reinitialize(pool_allocator=True,managed_memory=True)

torch.cuda.memory.change_current_allocator(
                                rmm_torch_allocator)
```

**Important Note (1)**: Make sure you add this code snippet before any other import in your Python script.

**Important Note (2)**: From a performance point of view, “spill” into CPU memory using UVM is not ideal!

**Important Note (3)**: Initialize GPU/CPU memory as pageable memory

# Hands-on

# Hands-on

Multiple options for all taste

**Demo**

- Peek at 4-way GH200 node
- Running NVbandwith

**Hands-on (see “practical2/README.md”)**

- Compile and run UVM experiments
- Install PyTorch with pip and containers
- Run simple CNN (MINST), adding UVM capabilities

**Bring Your Own Code (BYOC)**

- For AI: start creating your own python/conda environment under $SCRATCH
- For HPC: bootstrap a new spack environment, add dependencies by hand or via the tool, build your own app

